

CS 171 / EE 142: Intro to ML and DM

Christian Shelton

UC Riverside

Slide Set 18: Debugging Machine Learning



- From UC Riverside
 - ▶ CS 171 / EE 142: Introduction to Machine Learning and Data Mining
 - ▶ Professor Christian Shelton
- DO NOT REDISTRIBUTE
 - ▶ For use only by enrolled students in the course

Bugs or Errors in ML come from

- Incorrect code
- Incorrect approach

Telling the difference can be difficult.

Bugs or Errors in ML come from

- Incorrect code
- Incorrect approach

Telling the difference can be difficult.

Do not “randomly permute” your code and rerun.
There be madness.

Code debugging

Debugging like any other code:

- Identify and check invariants
- Test components
- Be explicit about interfaces

Classic: Gradient should point “uphill”

How to use:

- Write code to check and verify this.
- Execute verification at each gradient calculation.

Classic: Gradient should point “uphill”

How to use:

- Write code to check and verify this.
- Execute verification at each gradient calculation.

Other examples:

- If no data, regularizer will send all weights to zero
- Loss will always be positive (for some loss functions)
- If weights are zero, they will remain zero (for some methods)

Test Components

Examples:

- Vectorizing code:
 - ▶ Write simple “known correct” loop version
 - ▶ Write vectored version
 - ▶ Write wrapper that runs both and compares the answers
 - ▶ Use wrapper until you are sure vectored version is correct

Test Components

Examples:

- Vectorizing code:
 - ▶ Write simple “known correct” loop version
 - ▶ Write vectored version
 - ▶ Write wrapper that runs both and compares the answers
 - ▶ Use wrapper until you are sure vectored version is correct
- Use toy 2d data set:
 - ▶ Write plotting/drawing routines to visualize
 - ▶ Plot and “animate” algorithm
 - ▶ Verify that things are proceeding as expected

Test Components

Examples:

- Vectorizing code:
 - ▶ Write simple “known correct” loop version
 - ▶ Write vectored version
 - ▶ Write wrapper that runs both and compares the answers
 - ▶ Use wrapper until you are sure vectored version is correct
- Use toy 2d data set:
 - ▶ Write plotting/drawing routines to visualize
 - ▶ Plot and “animate” algorithm
 - ▶ Verify that things are proceeding as expected
- Extension of existing method
 - ▶ Set hyper-parameters of your method to reduce to existing method
 - ▶ Compare output to that of verified implementation of existing method

Be explicit about interfaces:

- Is “loss” is the loss or the negative loss?
- Is “grad” the gradient or negative gradient?
- “grad” is the gradient of what with respect to what?
- Does “foo” expect an example or can it take multiple examples?
- What array shapes are expected and what do the correspond to?

These will clarify your method in your own mind
and will make it easier to put the pieces together.

Thanks, Andrew

The remaining slides are based heavily on slides by **Andrew Ng**.

Dr. Ng kindly allowed me to use and adapt them.

Anti-spam Example

An example:

Anti-spam: Using bag-of-words (after feature selection down to 100 words),
classify e-mail as spam or not-spam.

Anti-spam Example

An example:

Anti-spam: Using bag-of-words (after feature selection down to 100 words),
classify e-mail as spam or not-spam.

Used logistic regression with L2 regularization:
20% test error

Anti-spam Example

An example:

Anti-spam: Using bag-of-words (after feature selection down to 100 words),
classify e-mail as spam or not-spam.

Used logistic regression with L2 regularization:
20% test error

Horrors! That's way too high!

What to do?

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features entirely (words in the header versus body of the e-mail)
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features entirely (words in the header versus body of the e-mail)
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

Don't try them all:

- It will take too long.
- You'll just be left wondering if you implemented any of those methods correctly.
- You won't really know if it worked, or you just got lucky.
- You won't learn anything for the next time you have this type of problem.

Diagnostic for bias-variance

Suppose you believe the problem is you have the wrong bias-variance trade-off.
That is, either:

- You are overfitting (high variance)
- You are underfitting (high bias)

What test would allow you to figure this out?

Diagnostic for bias-variance

Suppose you believe the problem is you have the wrong bias-variance trade-off.
That is, either:

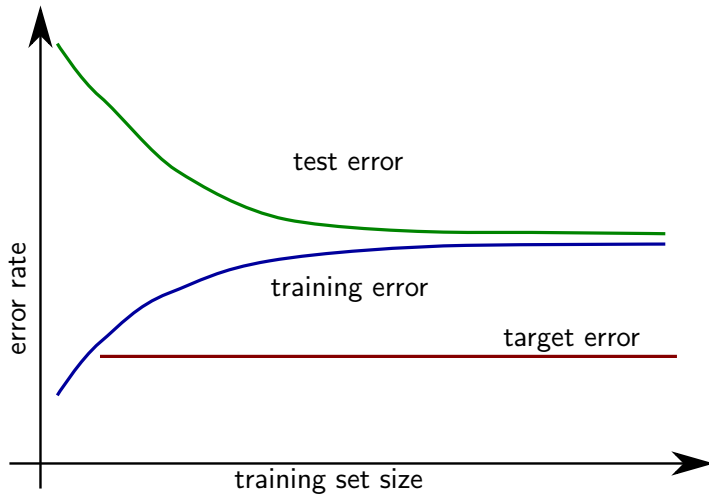
- You are overfitting (high variance)
- You are underfitting (high bias)

What test would allow you to figure this out?

Compare training error with test error.

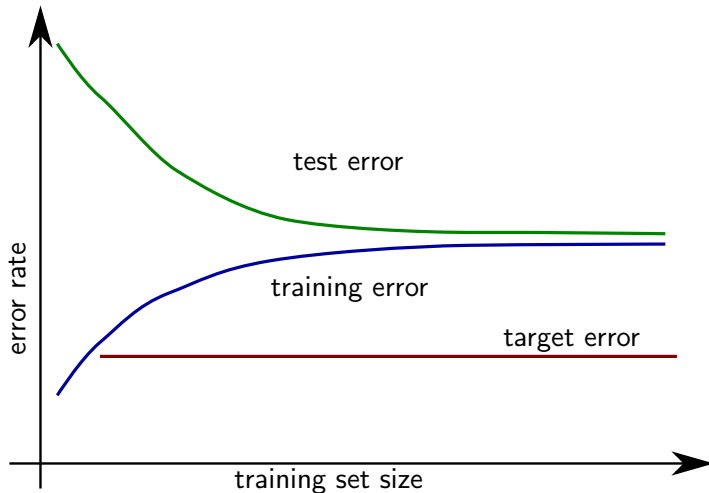
Even better: look how it changes with more training data

Diagnostic for bias-variance



Underfitting (high bias) or
overfitting (high variance)?

Diagnostic for bias-variance



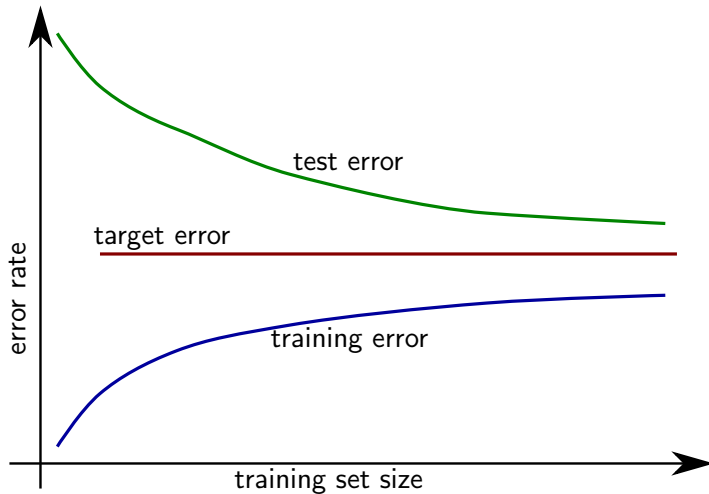
Underfitting (high bias) or
overfitting (high variance)?

underfitting (high bias)

Why?

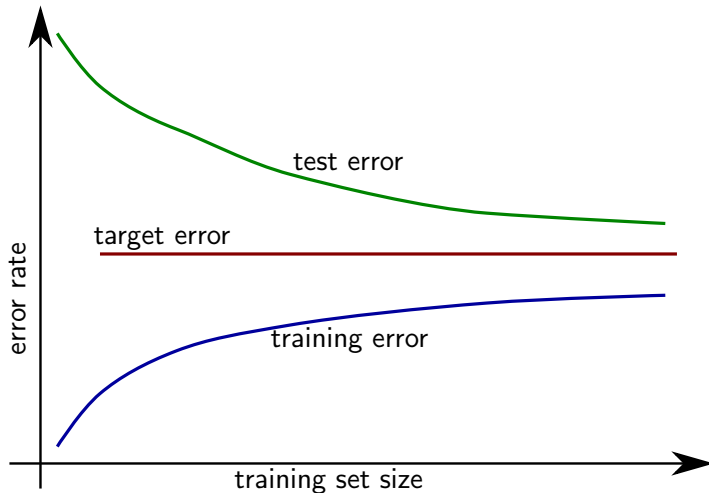
- even training error is too high
- small gap between test and training error

Diagnostic for bias-variance



Underfitting (high bias) or
overfitting (high variance)?

Diagnostic for bias-variance



Underfitting (high bias) or
overfitting (high variance)?

overfitting (high variance)

Why?

- test error continues to improve
- large gap between test and training error

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

fixes overfitting

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

fixes overfitting

fixes underfitting

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

fixes overfitting

fixes underfitting

fixes underfitting

You've improved your logistic regression error, but...

- LR gives 2% error on spam and 2% error on non-spam. (too high non-spam error!).
- SVM gives 10% error on spam and 0.01% error on non-spam. (acceptable).
- You'd prefer LR because of computational considerations (or something else).

Why?

You've improved your logistic regression error, but...

- LR gives 2% error on spam and 2% error on non-spam. (too high non-spam error!).
- SVM gives 10% error on spam and 0.01% error on non-spam. (acceptable).
- You'd prefer LR because of computational considerations (or something else).

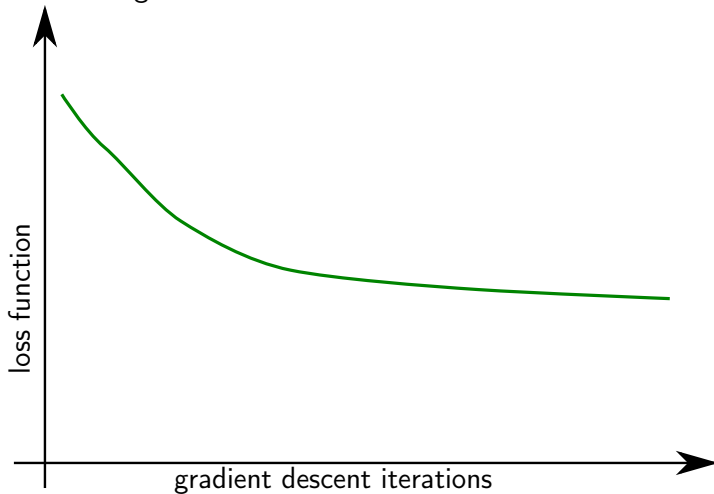
Why?

Two (of many) possible reasons

- LR optimization is halted before convergence
- Loss function for SVM is more appropriate

Optimization Diagnostics

Has this optimization converged?



Optimization Diagnostics

Are you optimizing the right function?

$$L_{\text{wt-01}}(w) = \sum_i \beta_i l_{01}(y_i f(x_i, w))$$

$$L_{\text{LR}}(w) = \sum_i l_{\text{lr}}(y_i f(x_i, w)) - \lambda w^\top w$$

$$L_{\text{SVM}}(w) = C \sum_i l_{\text{hinge}}(y_i f(x_i, w)) - w^\top w$$

The result of “optimizing” L_{LR} does worse than the result of “optimizing” L_{SVM} :

Optimization Diagnostics

Are you optimizing the right function?

$$L_{\text{wt-01}}(w) = \sum_i \beta_i l_{01}(y_i f(x_i, w))$$

$$L_{\text{LR}}(w) = \sum_i l_{\text{lr}}(y_i f(x_i, w)) - \lambda w^\top w$$

$$L_{\text{SVM}}(w) = C \sum_i l_{\text{hinge}}(y_i f(x_i, w)) - w^\top w$$

The result of “optimizing” L_{LR} does worse than the result of “optimizing” L_{SVM} :

$$w_{\text{LR}} = \underset{w}{\text{“arg min”}} L_{\text{LR}}(w)$$

$$w_{\text{SVM}} = \underset{w}{\text{“arg min”}} L_{\text{SVM}}(w)$$

$$L_{\text{wt-01}}(w_{\text{LR}}) > L_{\text{wt-01}}(w_{\text{SVM}})$$

Optimization Diagnostics

Are you optimizing the right function?

$$L_{\text{wt-01}}(w) = \sum_i \beta_i l_{01}(y_i f(x_i, w))$$

$$L_{\text{LR}}(w) = \sum_i l_{\text{lr}}(y_i f(x_i, w)) - \lambda w^\top w$$

$$L_{\text{SVM}}(w) = C \sum_i l_{\text{hinge}}(y_i f(x_i, w)) - w^\top w$$

The result of “optimizing” L_{LR} does worse than the result of “optimizing” L_{SVM} :

$$w_{\text{LR}} = \underset{w}{\text{“arg min”}} L_{\text{LR}}(w)$$

$$w_{\text{SVM}} = \underset{w}{\text{“arg min”}} L_{\text{SVM}}(w)$$

$$L_{\text{wt-01}}(w_{\text{LR}}) > L_{\text{wt-01}}(w_{\text{SVM}})$$

So what should we do?

Check $L_{\text{LR}}(w_{\text{LR}}) \leq L_{\text{LR}}(w_{\text{SVM}})$

Optimization Diagnostics

Check $L_{\text{LR}}(w_{\text{LR}}) \leq L_{\text{LR}}(w_{\text{SVM}})$

If $L_{\text{LR}}(w_{\text{LR}}) > L_{\text{LR}}(w_{\text{SVM}})$:

Then...

Check $L_{\text{LR}}(w_{\text{LR}}) \leq L_{\text{LR}}(w_{\text{SVM}})$

If $L_{\text{LR}}(w_{\text{LR}}) > L_{\text{LR}}(w_{\text{SVM}})$:

Then... optimization is not working. Try fixing that first.

Optimization Diagnostics

Check $L_{\text{LR}}(w_{\text{LR}}) \leq L_{\text{LR}}(w_{\text{SVM}})$

If $L_{\text{LR}}(w_{\text{LR}}) > L_{\text{LR}}(w_{\text{SVM}})$:

Then... optimization is not working. Try fixing that first.

If $L_{\text{LR}}(w_{\text{LR}}) < L_{\text{LR}}(w_{\text{SVM}})$:

Then...

Check $L_{\text{LR}}(w_{\text{LR}}) \leq L_{\text{LR}}(w_{\text{SVM}})$

If $L_{\text{LR}}(w_{\text{LR}}) > L_{\text{LR}}(w_{\text{SVM}})$:

Then... optimization is not working. Try fixing that first.

If $L_{\text{LR}}(w_{\text{LR}}) < L_{\text{LR}}(w_{\text{SVM}})$:

Then... optimization is (seems to be) working. Problem is with objective function.

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

fixes overfitting

fixes underfitting

fixes underfitting

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

fixes overfitting

fixes underfitting

fixes underfitting

fixes optimization algorithm

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

fixes overfitting

fixes underfitting

fixes underfitting

fixes optimization algorithm

fixes optimization algorithm

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

fixes overfitting

fixes underfitting

fixes underfitting

fixes optimization algorithm

fixes optimization algorithm

fixes optimization objective

Possible changes

We could

- Get more training data
- Reduce the set of features
- Increase the set of features
- Change the type of features
- Run gradient descent longer
- Use Newton's method to optimize
- Change λ
- Use SVMs instead

fixes overfitting

fixes overfitting

fixes underfitting

fixes underfitting

fixes optimization algorithm

fixes optimization algorithm

fixes optimization objective

fixes optimization objective

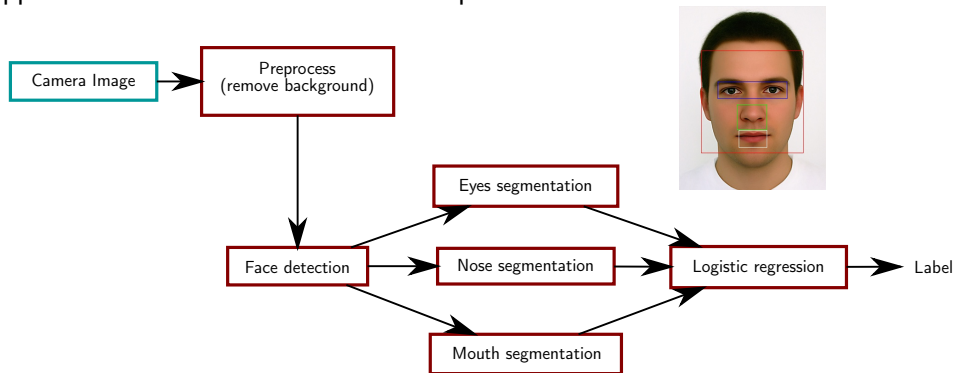
Error diagnostics

- Often you will need to come up with your own diagnostics.
- Understanding the basic principles of ML and of the algorithms will allow you to do this. (You cannot do this if you treat them as black boxes).

- Often you will need to come up with your own diagnostics.
- Understanding the basic principles of ML and of the algorithms will allow you to do this. (You cannot do this if you treat them as black boxes).
- Even if things are working well, you may still want to analyze your errors.
- Better to be able to explain why your method works, than just reporting that it does.
- You may be able to improve your results further if you understood why you are getting the results you are.

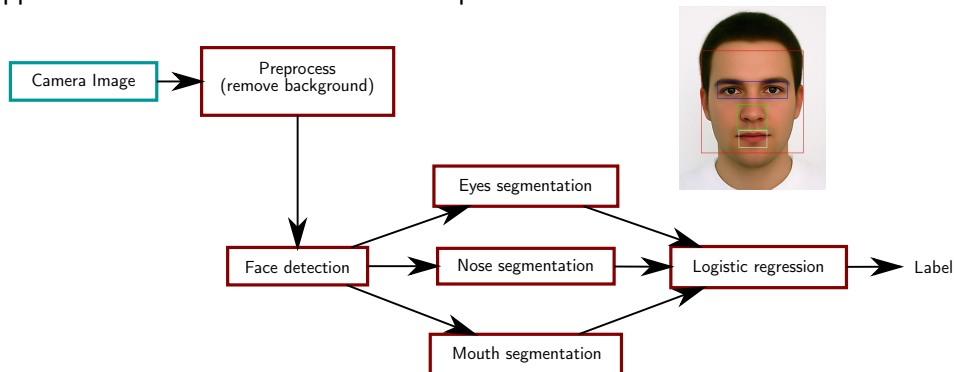
Error Analysis

Many applications combine different ML components:



Error Analysis

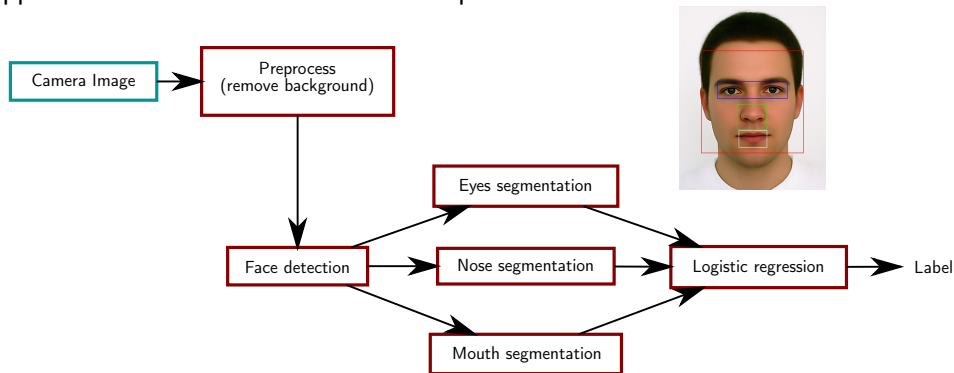
Many applications combine different ML components:



How much of your error comes from each component?

Error Analysis

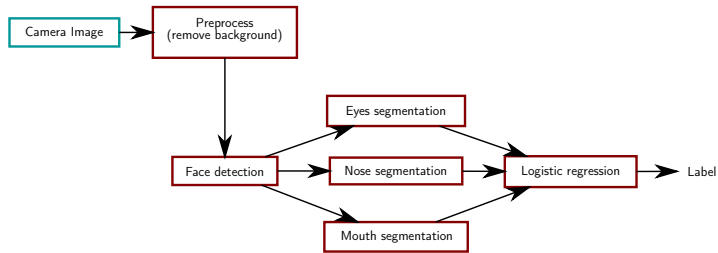
Many applications combine different ML components:



How much of your error comes from each component?

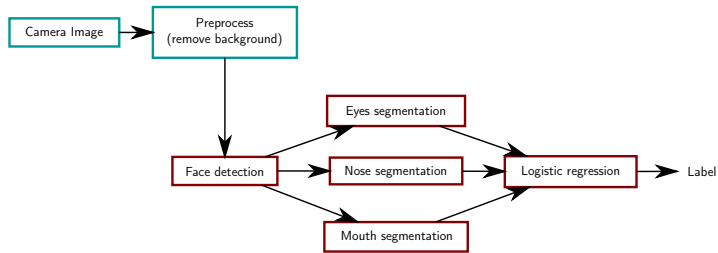
Plug in ground truth for each and see how much your error improves.

Error Analysis



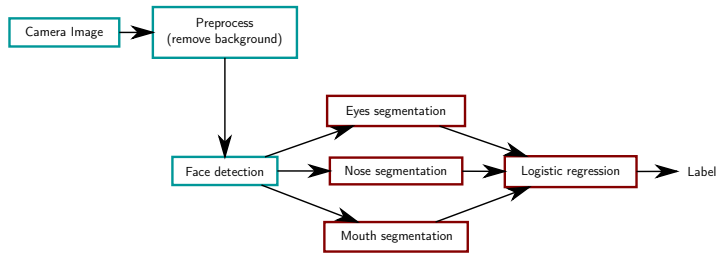
Component	Accuracy
Overall	85%
Preprocess	
Face Det.	
Eyes Seg.	
Nose Seg.	
Mouth Seg.	
LR	

Error Analysis



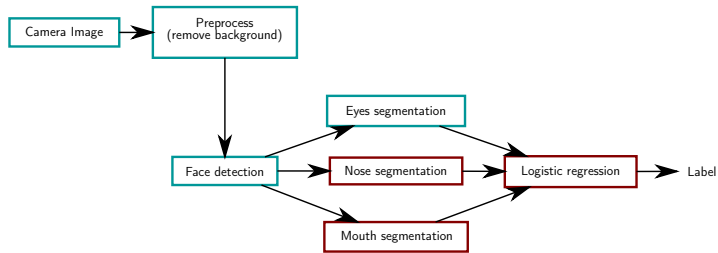
Component	Accuracy
Overall	85%
Preprocess	86%
Face Det.	
Eyes Seg.	
Nose Seg.	
Mouth Seg.	
LR	

Error Analysis



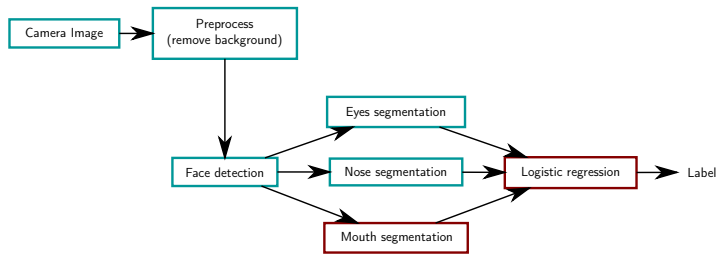
Component	Accuracy
Overall	85%
Preprocess	86%
Face Det.	91%
Eyes Seg.	
Nose Seg.	
Mouth Seg.	
LR	

Error Analysis



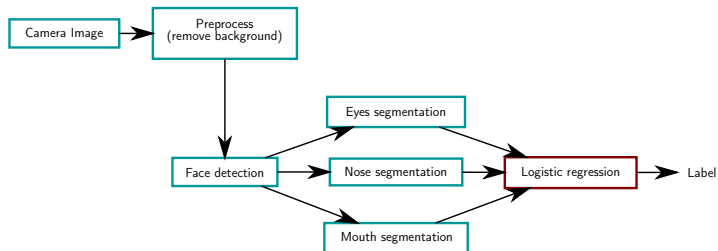
Component	Accuracy
Overall	85%
Preprocess	86%
Face Det.	91%
Eyes Seg.	95%
Nose Seg.	
Mouth Seg.	
LR	

Error Analysis



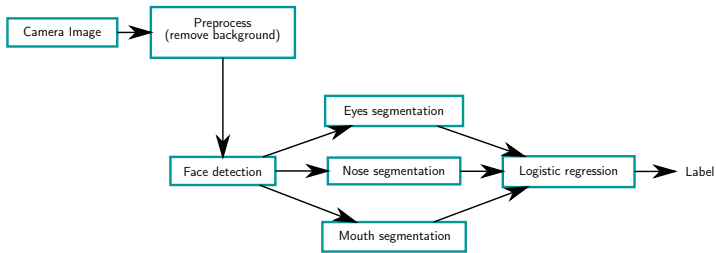
Component	Accuracy
Overall	85%
Preprocess	86%
Face Det.	91%
Eyes Seg.	95%
Nose Seg.	96%
Mouth Seg.	
LR	

Error Analysis



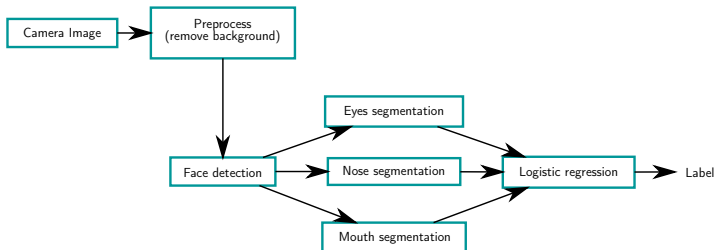
Component	Accuracy
Overall	85%
Preprocess	86%
Face Det.	91%
Eyes Seg.	95%
Nose Seg.	96%
Mouth Seg.	97%
LR	

Error Analysis



Component	Accuracy
Overall	85%
Preprocess	86%
Face Det.	91%
Eyes Seg.	95%
Nose Seg.	96%
Mouth Seg.	97%
LR	100%

Error Analysis



Component	Accuracy
Overall	85%
Preprocess	86%
Face Det.	91%
Eyes Seg.	95%
Nose Seg.	96%
Mouth Seg.	97%
LR	100%

So, Face Detection and Eye Segmentation could use the most help.

Consider a spam classifier that has many clever features (moving it from 94% to 99.9% accuracy!):

- spelling correction
- sender host features
- e-mail header features
- e-mail text parser features
- javascript parser
- embedded image features

How much did each help?

Ablative Analysis

Consider a spam classifier that has many clever features (moving it from 94% to 99.9% accuracy!):

- spelling correction
- sender host features
- e-mail header features
- e-mail text parser features
- javascript parser
- embedded image features

How much did each help?

Remove them one at a time and see the performance:

Component	Accuracy
Overall	99.9%
Spelling	99.0%
Host Feat.	98.9%
Header Feat.	98.9%
Text Parser	95.0%
Javascript	94.5%
Image Feat.	94.0%

Ablative Analysis

Consider a spam classifier that has many clever features (moving it from 94% to 99.9% accuracy!):

- spelling correction
- sender host features
- e-mail header features
- e-mail text parser features
- javascript parser
- embedded image features

How much did each help?

Remove them one at a time and see the performance:

Component	Accuracy
Overall	99.9%
Spelling	99.0%
Host Feat.	98.9%
Header Feat.	98.9%
Text Parser	95.0%
Javascript	94.5%
Image Feat.	94.0%

So, e-mail text parser is the most helpful component

How to Start?

Do

- Plot your data.

How to Start?

Do

- Plot your data.
- Build a prototype and analyze it.

How to Start?

Do

- Plot your data.
- Build a prototype and analyze it.

How to Start?

Do

- Plot your data.
- Build a prototype and analyze it.

Don't

- Start by designing a system to handle everything you can think of.

How to Start?

Do

- Plot your data.
- Build a prototype and analyze it.

Don't

- Start by designing a system to handle everything you can think of.
- Be afraid to start over.

How to Start?

Do

- Plot your data.
- Build a prototype and analyze it.

Don't

- Start by designing a system to handle everything you can think of.
- Be afraid to start over.
- Shave yaks.